11 · Exp 12 — Circular linked list (insert, delete, traverse)

Exp 11 with one change: the last node points back to head instead of NULL. Every "== NULL" test becomes "== head".

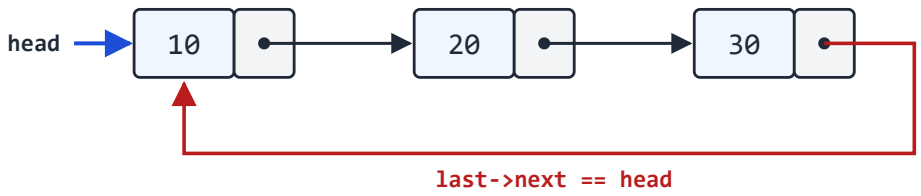
Exp 11 list → **last node points to head** → find last: next == head → traverse: do-while → delete first: fix the last node

11.1 The idea

GUESS FIRST

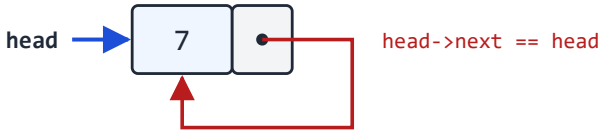
In a circular list there is no NULL to stop at. If traverse used `while (temp != head)` with `temp = head` at the start, what would it print?

Circular: the last node points back to head (never NULL)



Traverse with do { ... } while (temp != head); - a plain while (temp != head) would stop before printing anything.

The manual's list after inserting 10, 20, 30.



A single node points to itself. The program must treat this case separately.

	Singly (Exp 11)	Circular (Exp 12)
last node's next	NULL	head
"is this the last node?"	`temp->next == NULL`	`temp->next == head`
traverse loop	`while (temp != NULL)`	`do { … } while (temp != head);`
delete from beginning	O(1)	**O(n)**: the last node must be found and re-pointed to the new head

The answer to the guess: **nothing**. temp starts *equal* to head, so a `while` loop exits before its first pass. A `do … while` prints first and tests afterwards.

11.2 The program (corrected)

The manual's PROGRAM listing doesn't compile:

G++ SAYS

```
m12_listing.cpp:62:17: error: missing terminating " character
   62 |     {   cout << "\n1. Insert
      |                 ^
m12_listing.cpp:13:6: error: variable or field 'insert' declared void
   13 | void insert(intvalue)
      |      ^~~~~~
m12_listing.cpp:13:13: error: 'intvalue' was not declared in this scope
   13 | void insert(intvalue)
      |             ^~~~~~~~
m12_listing.cpp:46:15: error: unused variable 'temp' [-Werror=unused-variable]
   46 |     {   Node* temp = head;
      |               ^~~~
m12_listing.cpp:55:16: error: 'temp' was not declared in this scope; did you mean 'tm'?
   55 |     }   delete temp;
      |                ^~~~
m12_listing.cpp:62:14: error: no match for 'operator<<' (operand types are 'std::ostream' {aka
'std::basic_ostream<char>'} and 'std::ostream' {aka 'std::basic_ostream<char>'})
   62 |     {   cout << "\n1. Insert
      |         ~~~~ ^~
```

The four problems: `insert(intvalue)` (missing space); the menu `cout` lines are missing their closing `" << endl;` ; `delete temp;` sits after the closing `}` of the block where `temp` was declared; and `main()` is cut off at the page end. The IMPLEMENTATION screenshot (page 7) is correct and complete; this pack follows it.

E12_CIRCULAR_LINKED_LIST.CPP

```cpp
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *next;
};

Node *head = NULL;

// Insert at the end
void insert(int value)                    // FIX: manual printed "intvalue"
{
    Node *newNode = new Node;
    newNode->data = value;

    if (head == NULL)
    {
        head = newNode;
        newNode->next = head;             // one node points to itself
    }
    else
    {
        Node *temp = head;
```

```
        while (temp->next != head)          // last node is the one pointing to head
            temp = temp->next;
        temp->next = newNode;
        newNode->next = head;               // close the circle
    }
    cout << value << " inserted into list." << endl;
}

// Delete from the beginning
void deleteNode()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else if (head->next == head)            // only one node
    {
        cout << head->data << " deleted from list." << endl;
        delete head;
        head = NULL;
    }
    else
    {
        Node *temp = head;
        Node *last = head;
        while (last->next != head)          // find the last node
            last = last->next;

        cout << head->data << " deleted from list." << endl;
        head = head->next;
        last->next = head;                  // last now points to the new head
        delete temp;                        // FIX: was outside this block (temp unknown there)
    }
}

void traverse()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else
    {
        Node *temp = head;
        cout << "Circular Linked List: ";
        do                                  // do-while: print head before testing
        {
            cout << temp->data << " ";
            temp = temp->next;
        } while (temp != head);
        cout << endl;
    }
}

int main()
{
    int choice, value;

    do
    {
        cout << "\n1. Insert" << endl;       // FIX: manual's menu strings were unclosed
```

```
        cout << "2. Delete" << endl;
        cout << "3. Traverse" << endl;
        cout << "4. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter value to insert: ";
            cin >> value;
            insert(value);
        }
        else if (choice == 2)
        {
            deleteNode();
        }
        else if (choice == 3)
        {
            traverse();
        }
        else if (choice == 4)
        {
            cout << "Program terminated." << endl;
        }
        else
        {
            cout << "Invalid choice!" << endl;
        }
    } while (choice != 4);

    return 0;
}
```

The manual's sample run, reproduced exactly:

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: 1
Enter value to insert: 10
10 inserted into list.
Enter your choice: 1
Enter value to insert: 20
20 inserted into list.
Enter your choice: 1
Enter value to insert: 30
30 inserted into list.
Enter your choice: 3
Circular Linked List: 10 20 30
Enter your choice: 2
10 deleted from list.
Enter your choice: 3
Circular Linked List: 20 30
Enter your choice: 4
Program terminated.
```

Deleting the only node (the `head->next == head` branch), then deleting from an empty list:

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: 1
Enter value to insert: 7
7 inserted into list.
Enter your choice: 2
7 deleted from list.
Enter your choice: 3
List is empty.
Enter your choice: 2
List is empty.
Enter your choice: 4
Program terminated.
```

DELETE-FROM-BEGINNING, THREE CASES

1. **Empty** (`head == NULL`): print a message.
2. **One node** (`head->next == head`): delete it, set `head = NULL` . Without this case, the general code would set head to the deleted node itself.
3. **General:** find `last` (its next is head), move `head = head->next` , set `last->next = head` , then `delete temp` .

WRITE IN YOUR RECORD

Aim: To implement a circular linked list and perform insertion, deletion and traversal.

Algorithm: *Insert (end):* create node; if head == NULL: head = node, node->next = head; else move temp until temp->next == head, temp->next = node, node->next = head. *Delete (beginning):* if empty → message; if one node → delete it, head = NULL; else find last, temp = head, head = head->next, last->next = head, delete temp. *Traverse:* temp = head; do { print temp->data; temp = temp->next } while (temp ≠ head).

Complexity: insert at end O(n), delete at beginning O(n) (finding the last node), traverse O(n); space O(n), auxiliary O(1).

Result: Inserted 10, 20, 30 (list: 10 20 30); deleted 10 (list: 20 30). The last node always pointed back to head.

11.3 Viva questions

Question	Answer
What is a circular linked list?	A linked list whose last node points to the first node instead of NULL.
How do you detect the last node?	`temp->next == head`.
Why `do-while` for traversal?	The stop condition (`temp == head`) is already true at the start; do-while prints head first.
Why is delete-from-beginning O(n) here but O(1) in Exp 11?	The last node's next must be updated to the new head, and finding it takes a walk. (A `tail` pointer would make it O(1).)
One-node list: what is head->next?	head itself.
Advantages?	Any node can reach every other node; natural for round-robin scheduling, multiplayer turns, repeating playlists.
Risk?	Infinite loops, if a traversal's stop condition is wrong.
Types?	Circular singly linked (next only) and circular doubly linked (next and prev).

11.4 If the examiner changes the question

The same assignment list as Exp 11, adapted so the circle stays closed (10 checks, all pass, each including "is it still circular?").

Insert at the beginning. The last node must point to the new head:

```
// Like insert at end, but afterwards head moves to the new node
void insertBegin(int value)
{
    Node *newNode = new Node;
    newNode->data = value;
    if (head == NULL)
    {
        head = newNode;
        newNode->next = head;
        return;
    }
    Node *last = head;
    while (last->next != head)              // the last node must point to the new first node
        last = last->next;
    newNode->next = head;
    last->next = newNode;
    head = newNode;
}
```

Delete from the end:

```
void deleteEnd()
{
    if (head == NULL)
    {
```

```
        cout << "List is empty." << endl;
        return;
    }
    if (head->next == head)                 // only one node
    {
        cout << head->data << " deleted from list." << endl;
        delete head;
        head = NULL;
        return;
    }
    Node *temp = head;
    while (temp->next->next != head)        // stop at the second-last node
        temp = temp->next;
    cout << temp->next->data << " deleted from list." << endl;
    delete temp->next;
    temp->next = head;                      // new last node closes the circle
}
```

Search and count (do-while again):

```
int search(int key)                         // 1-based position, -1 if absent
{
    if (head == NULL)
        return -1;
    Node *temp = head;
    int pos = 1;
    do
    {
        if (temp->data == key)
            return pos;
        temp = temp->next;
        pos++;
    } while (temp != head);
    return -1;
}
```

```
int countNodes()
{
    if (head == NULL)
        return 0;
    int count = 0;
    Node *temp = head;
    do
    {
        count++;
        temp = temp->next;
    } while (temp != head);
    return count;
}
```

Display in reverse (recursion that stops when the next node is head):

```
// Recursion that stops when it comes back round to head
void reverseFrom(Node *p)
{
    if (p->next != head)
        reverseFrom(p->next);
    cout << p->data << " ";
}

void displayReverse()
{
    if (head == NULL)
        cout << "List is empty.";
    else
        reverseFrom(head);
    cout << endl;
}
```

Insert / delete at a position:

```
void insertAtPos(int pos, int value)         // pos 1 = new first node
{
    int n = countNodes();
    if (pos < 1 || pos > n + 1)
    {
        cout << "Invalid position." << endl;
        return;
    }
    if (pos == 1)
    {
        insertBegin(value);
        return;
    }
    Node *temp = head;
    for (int i = 1; i < pos - 1; i++)        // go to node pos-1
        temp = temp->next;
    Node *newNode = new Node;
    newNode->data = value;
    newNode->next = temp->next;              // for pos = n+1 this is head: circle stays closed
    temp->next = newNode;
}
```

```
void deleteAtPos(int pos)
{
    int n = countNodes();
    if (pos < 1 || pos > n)
    {
        cout << "Invalid position." << endl;
        return;
    }
    if (pos == 1)                            // same as Exp 12's deleteNode()
    {
        Node *last = head;
        while (last->next != head)
            last = last->next;
        Node *del = head;
        cout << del->data << " deleted from list." << endl;
        if (n == 1)
            head = NULL;
        else
```

```
        {
            head = head->next;
            last->next = head;
        }
        delete del;
        return;
    }
    Node *temp = head;
    for (int i = 1; i < pos - 1; i++)
        temp = temp->next;
    Node *del = temp->next;
    temp->next = del->next;
    cout << del->data << " deleted from list." << endl;
    delete del;
}
```

CHECK 11.1

Circular list head → 3 → 6 → 9 → (back to 3). Run delete-from-beginning by hand: what are `last`, the new head, and `last->next` afterwards?

CHECK 11.2

Why does `insert` in Exp 12 need `newNode->next = head;` even in the `head == NULL` branch?

Answers

Check 11.1: last = node 9; head moves to node 6; `last->next = head` → 9 now points to 6; node 3 is deleted. List: 6 → 9 → (back to 6).

Check 11.2: A lone node is both first and last, so its next must point to head (itself) to keep the list circular. If it were left unset, the next traversal or insert would follow a garbage address.